

SOUL: Sparse Feature Policy Unlearning

Mitigates State Hallucination in Vision-Language-Action Models

Anonymous authors

Abstract—Vision-Language-Action models (VLAs) have shown strong generalization in robotic manipulation by leveraging rich representations from pre-trained vision-language models. However, their deployment in real-world environments remains limited by recurring unreliable behaviors. In this work, we study state hallucination, a recurring failure pattern in which a VLA continues acting that has not actually been achieved. To diagnose this phenomenon, we find that state hallucination coincides with weakened attention on task-relevant visual regions, and a mechanistic interpretation via sparse autoencoders reveals that hallucination-associated sparse features emerge and are activated when these failures occur. Based on this analysis, we propose SOUL (Sparse feature pOlicy UnLearning), which selectively unlearns policy knowledge associated with state hallucination behaviors by defining hallucination- and nominal-associated sparse features as explicit forgetting and retention targets. Experiments across VLA architectures in simulated and real-world environments show that our method substantially reduces hallucinated failures and improves task success without compromising the original capability. These results suggest that interpretable feature analysis provides a practical basis for selectively modifying undesirable knowledge in robot policies.

I. INTRODUCTION

Vision-language-action models (VLAs) have emerged as a promising approach for bringing foundation models into robot manipulation [1], [2], [3]. By integrating visual and language understanding with action generation, VLAs can translate high-level instructions into continuous robot control. Large-scale pretraining on heterogeneous robot and vision-language data has further enabled generalization across tasks, objects, environments, and robot embodiments [4], [5], [6]. Despite this progress, reliable VLA execution remains challenging, with recent studies reporting failures such as unstable and premature actions, long-horizon drift, limited failure recovery, and infeasible actions [7], [8], [9], [10], [11], [12], [13]. This is particularly concerning in physical robot execution, where erroneous actions directly affect the robot and its environment.

Within this broader set of failures, we focus on a recurring failure pattern in which a VLA continues a manipulation sequence based on a robot-object state that has not actually been achieved. We refer to this behavioral pattern as **state hallucination**, because the policy acts as if an unrealized physical state were true. For example, a VLA may close its gripper before reaching an object and subsequently execute transport and place motions as though the object had been successfully grasped, as illustrated in the upper callout of Fig. 1(a). Such a mismatch can persist across subsequent actions, allowing a local execution error to propagate over multiple stages. This recurring and persistent behavior motivates us to investigate whether policy knowledge associated

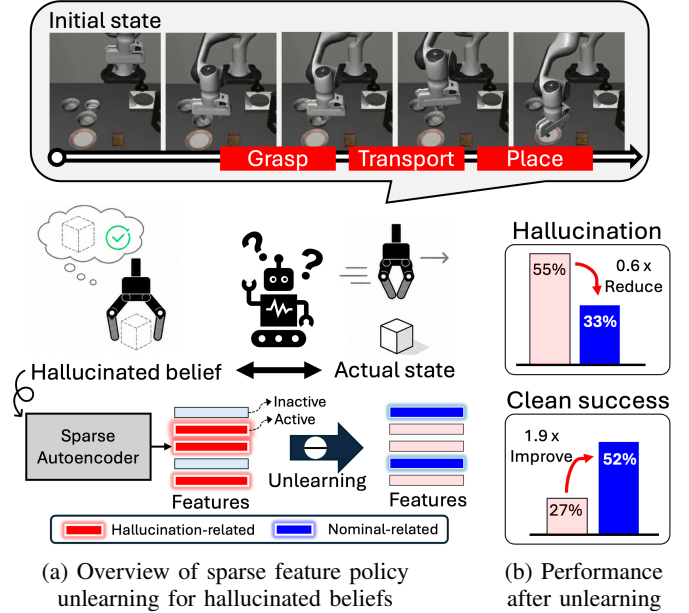


Fig. 1: **Sparse feature policy unlearning mitigates hallucinated beliefs in VLAs.** (a) The top callout shows examples of grasp, transport, and place state hallucinations. State hallucination occurs when a VLA mistakes its hallucinated belief for the actual robot-object state and generates actions based on this false belief. Hallucination-related features activated during this mismatch are suppressed through the proposed unlearning while preserving nominal-related features. (b) Our sparse feature policy unlearning reduces hallucination while improving clean (hallucination-free) task success.

with state hallucination can be selectively unlearned while preserving successful manipulation capabilities.

To this end, we first examine how the policy’s internal representations differ between state hallucination and successful executions. Such hallucination rollouts exhibit weakened attention to task-relevant regions, such as the gripper and target object, together with more spatially dispersed attention. Through a mechanistic analysis with sparse autoencoders, we identify sparse features whose activations are strongly associated with hallucination in specific visual regions. Based on these observations, we propose *Sparse feature pOlicy UnLearning (SOUL)*, which selectively unlearns policy knowledge associated with state hallucination by using hallucination- and nominal-associated region-feature pairs as selective forgetting and retention targets, respectively. This feature-level policy unlearning objective aims to mitigate

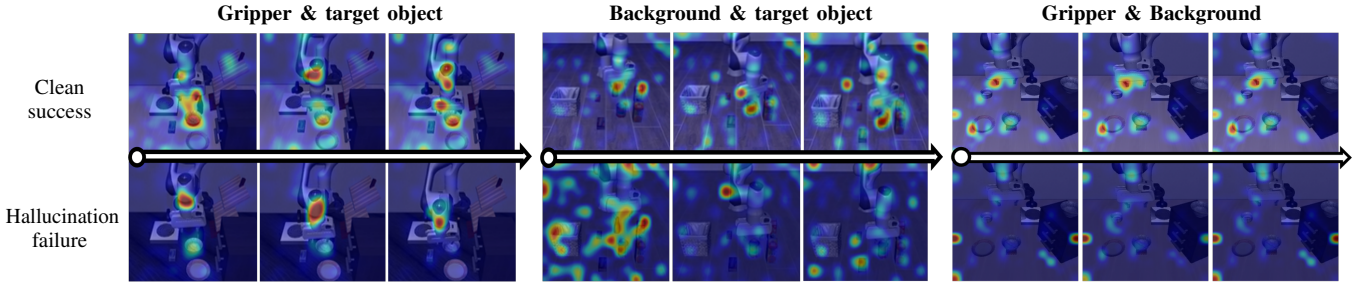


Fig. 2: **State hallucination is associated with distinct attention patterns.** Representative clean success (CS) and hallucination failure (HF) rollouts under the same task instruction show distinct attention patterns. HF exhibit weaker attention to task-relevant regions (e.g., gripper and target object) and increased attention to the background compared with CS.

recurring hallucination behavior while minimizing interference with useful manipulation capabilities. We validate our approach with OpenVLA [2] and $\pi_{0.5}$ [3] on LIBERO-Plus [9], RoboCasa [14], and a real-world Franka robot, achieving an average 21.6% reduction in hallucination rate and a 22.2% improvement in task success while preserving an average of 89% of previously successful behaviors.

Our contributions are threefold. First, we characterize state hallucination as a recurring failure pattern in which a VLA continues acting as though an unrealized robot-object state had been achieved. Second, we analyze state hallucination through visual attention and sparse features, revealing weakened attention to task-relevant regions and distinct feature patterns associated with state hallucination. Third, we introduce a sparse feature policy unlearning method that selectively unlearns recurring state hallucination behavior by suppressing hallucination-associated features while preserving those that support successful manipulation. Overall, our results suggest that mechanistic feature analysis can reveal internal representations associated with recurring policy failures, providing a basis for selectively unlearning such behaviors in embodied foundation models.

II. RELATED WORK

A. Vision-Language-Action Models

Vision-language-action models have recently become a major direction for generalist robot manipulation. One major line of work improves generalization by scaling pretrained vision-language representations, heterogeneous robot demonstration data, and action-generation architectures [1], [2], [3], [4], [5], [6], [15], [16], [17], [18]. Another line of work improves deployment reliability through intermediate reasoning [19], [20], failure detection and recovery [12], [21], and test-time sampling or verification [22], [23]. These approaches improve action generation, selection, or recovery during execution, but do not directly modify the internal representations associated with recurring failure behaviors. Recent concurrent mechanistic interpretability studies have begun to analyze and steer VLA behavior by intervening on internal representations at inference time, including feed-forward activation directions [24] and sparse features [25].

In contrast, we formulate the selective modification of undesirable policy behavior as a policy unlearning problem,

where internal knowledge associated with recurring hallucination is removed while representations supporting successful execution are preserved. By incorporating this modification into the pre-trained policy parameters, our approach enables persistent behavior correction without repeated feature-level intervention at inference time.

B. Machine Unlearning

Machine unlearning aims to make a trained model forget the knowledge associated with selected data without retraining the model from scratch [26], [27], [28], [29]. Prior work has studied this problem mainly in classification tasks, where selected examples are removed, and in language model tasks, where factual knowledge or undesired responses are suppressed [30], [31]. A central challenge is balancing the removal of undesirable behavior with the preservation of useful model capabilities. This trade-off is particularly consequential for robot policies, where modifying one execution behavior can disrupt tightly coupled various manipulation skills. We therefore formulate state hallucination mitigation as selective policy unlearning, reducing the influence of hallucination-associated knowledge while preserving other knowledge that support successful normal execution.

C. Sparse Autoencoders

Sparse autoencoders (SAEs) decompose dense model activations into sparse latent representations, enabling individual features to be associated with more localized and interpretable patterns in model behavior [32], [33]. They are trained to reconstruct the original activations under a sparsity constraint, such that only a small subset of latent features is active for each input. A commonly used variant is the Top- k SAE, which imposes this sparsity constraint explicitly by retaining only the k largest latent activations. In VLAs, SAE features have recently been used to interpret and steer policy at inference time [25]. In contrast, we use sparse features to characterize recurring hallucination patterns and define explicit forgetting and retention targets for selective policy unlearning.

III. WHAT IS STATE HALLUCINATION IN VLAs?

A. Definition

We define **state hallucination** as an execution pattern in which a VLA continues to generate actions based on a

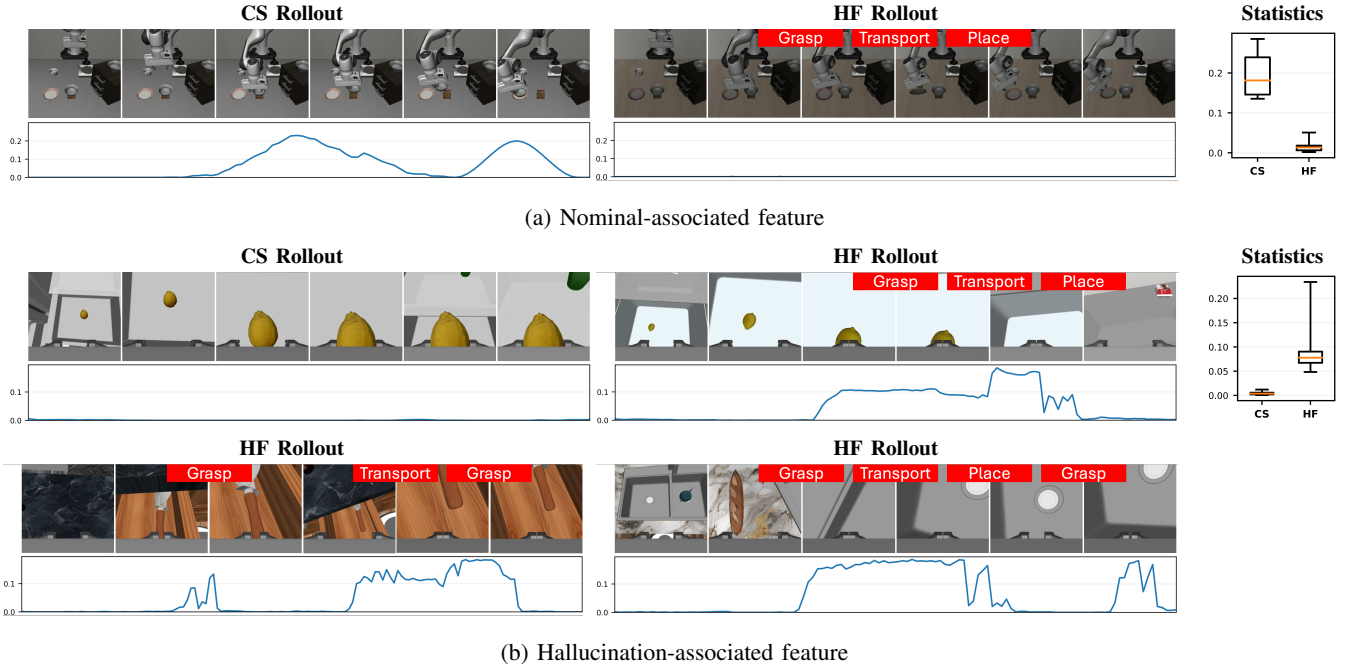


Fig. 3: **Sparse features associated with state hallucination can be identified.** For each feature, we visualize representative temporal activations, $m_{e,r,j}$ defined in Section III-C, and its statistics across hallucination-free success, clean success (CS), and failure with hallucination, hallucination failure (HF), rollouts. (a) A nominal-associated feature is strongly activated when the policy successfully executes the corresponding subtask or completes the overall task, while remaining largely inactive in HF rollouts. (b) A hallucination-associated feature is strongly activated when state hallucination occurs during grasp, transport, or place. The boxplots further show that the activation differences between CS and HF are consistent across rollouts.

robot-object state that has not actually been achieved. State hallucination can occur at different manipulation stages.

- A *grasp* hallucination occurs when the gripper closes before physically contacting the target object. Grasp failures involving object contact, such as inaccurate localization or slippage, are not considered hallucinations.
- A *transport* hallucination occurs when, after an unsuccessful grasp, the gripper moves toward the destination while the object remains ungrasped.
- A *place* hallucination occurs when the gripper opens at the destination without holding the target object.

We label these events using ground-truth robot-object states in simulation and observable interaction states in real-world rollouts. Based on these criteria, we categorize each rollout as clean success (CS), hallucination failure (HF), or non-hallucination failure (NF). Here, HF denotes failed executions exhibiting state hallucination, while CS and NF denote successful and failed executions without state hallucination.

B. Analysis of Normal and State Hallucinated Executions

To examine how state hallucination is reflected in the policy, we compare internal attention patterns between CS and HF executions. At each rollout step, we aggregate the attention from action tokens to visual tokens and project the resulting attention scores onto the input image, revealing which visual regions the policy attends to while generating actions. For visualization, we show representative time steps

immediately preceding the corresponding manipulation event, such as grasping. As shown in Fig. 2, CS executions exhibit attention concentrated on task-relevant regions, particularly the target object and gripper. In contrast, HF executions show weaker attention to these regions and increased attention to background regions. These differences suggest that state hallucination is accompanied by a distinct shift in the policy’s visual attention. This motivates us to look beyond attention and examine whether hallucinated executions are also reflected in the internal representations of the policy.

C. Mechanistic Interpretation of State Hallucination

To investigate the internal representations associated with state hallucination, we analyze VLA activations using sparse autoencoders (SAEs) [32]. SAEs decompose dense representations into sparse features, allowing us to compare feature patterns across execution outcomes, such as clean success, hallucination failure, and non-hallucination failure.

SAE training details. To extract sparse features from policy activations, we collect visual-token activations from the residual stream transformer layers throughout each rollout and group them by execution outcome. The visual token activations allow sparse features to be examined with respect to semantic scene regions. For an input sample x , let $x_p \in \mathbb{R}^d$ denote the activation corresponding to visual token p . We flatten the collected activations across rollout steps and visual tokens, treating each x_p as an individual SAE training sample. The encoder maps x_p to a latent representation and

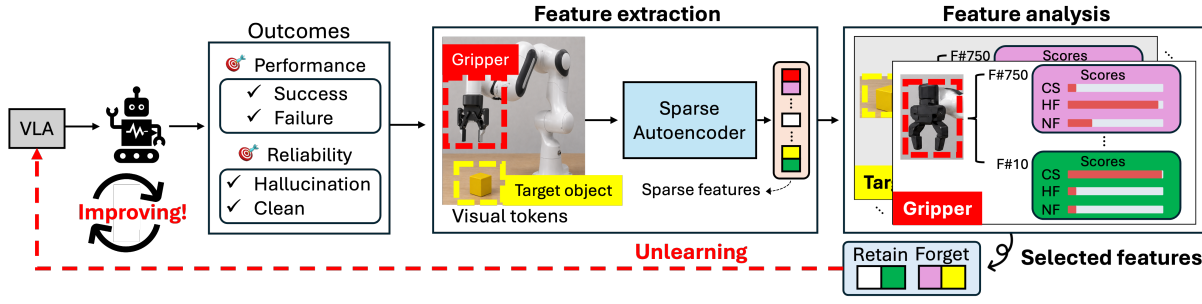


Fig. 4: **Overall framework.** The proposed method provides a general framework for improving robot policies by identifying and selectively modifying internal representations associated with undesirable behaviors. It first extracts visual-region-specific sparse features from policy activations and analyzes their activation patterns across execution outcomes, including clean success (CS), hallucination failure (HF), and non-hallucination failure (NF), characterized by performance and reliability. Based on these associations, region-feature pairs are selected as forget or retain targets. The policy is then updated to suppress hallucination-associated features while preserving reliable behavior. In this way, the policy is progressively improved.

retains only the k largest positive activations, yielding a sparse feature vector $z_p(x)$, whose j -th element $z_{p,j}(x)$ denotes the activation of SAE feature j at visual token p . The decoder reconstructs the original policy activation x_p from $z_p(x)$. After training, we freeze the SAE and use its encoder to extract sparse feature activations for subsequent analysis.

Feature association analysis. Motivated by the region-level attention differences observed in Section III-B, we characterize sparse features with respect to the semantic regions in which they are activated. We can obtain semantic regions from the masks provided by the simulation benchmarks, while manually annotating the corresponding regions for real-world observations. For an input sample x , let $\mathcal{R}_r(x)$ denote the set of visual tokens belonging to region r . We define the region-conditioned feature activation as

$$z_{r,j}(x) = \frac{1}{|\mathcal{R}_r(x)|} \sum_{p \in \mathcal{R}_r(x)} z_{p,j}(x). \quad (1)$$

This measures how strongly feature j is expressed within semantic region r . For rollout e at time step t , we denote the corresponding region-conditioned feature activation by $z_{e,t,r,j}$. Since hallucination-associated features may become active only during a short period of a rollout, we summarize each region-feature pair by its maximum activation over time as $m_{e,r,j} = \max_t z_{e,t,r,j}$. We then average the rollout-level activation within each execution outcome. Let $\mathcal{E}_c$ denote the set of rollouts belonging to outcome $c \in \{\text{CS}, \text{NF}, \text{HF}\}$. The outcome-level activation is computed as

$$\bar{m}_{c,r,j} = \frac{1}{|\mathcal{E}_c|} \sum_{e \in \mathcal{E}_c} m_{e,r,j}. \quad (2)$$

For each region-feature pair, we define hallucination-associated and nominal-associated scores by contrasting its activation in each condition against the other two conditions as

$$\begin{aligned} s_{r,j}^{\text{hall}} &= \bar{m}_{\text{HF},r,j} - \frac{1}{2} (\bar{m}_{\text{CS},r,j} + \bar{m}_{\text{NF},r,j}), \\ s_{r,j}^{\text{nom}} &= \bar{m}_{\text{CS},r,j} - \frac{1}{2} (\bar{m}_{\text{HF},r,j} + \bar{m}_{\text{NF},r,j}). \end{aligned} \quad (3)$$

Here, a larger $s_{r,j}^{\text{hall}}$ indicates that feature j at region r is selectively activated in hallucination failures compared with the other groups. Conversely, a larger $s_{r,j}^{\text{nom}}$ indicates stronger specificity to clean successful executions.

Sparse feature analysis. Fig. 3 visualizes representative hallucination-associated and nominal-associated features across HF and CS rollouts. The hallucination-associated feature exhibits strong activation around state hallucination events in HF, while remaining relatively weak in CS. Conversely, the nominal-associated feature is activated during successful subtask execution or task completion in CS, while remaining weaker in HF. We further compare their rollout-level activation distributions across CS and HF, showing consistent activation differences across multiple tasks and visual conditions beyond the representative trajectories. These results indicate that the identified sparse features provide interpretable representations of behaviorally meaningful patterns associated with state hallucination and clean execution. More visualizations are in Section V-B and supplementary materials.

IV. SPARSE FEATURE POLICY UNLEARNING (SOUL)

Building on the sparse feature analysis in Section III, we aim to mitigate state hallucination while preserving normal capabilities. To this end, we formulate this problem as selective policy unlearning, where internal representations associated with harmful state hallucination behavior are selectively suppressed. Based on this formulation, we use region-conditioned sparse features as explicit forgetting and retention targets for policy unlearning, as illustrated in Fig. 4.

Feature selection for forgetting and retention. We construct feature sets for forgetting and retention based on the feature association scores defined in Section III-C. To further isolate features specific to hallucination failures, we introduce a separation score $s_{r,j}^{\text{sep}} = \bar{m}_{\text{HF},r,j} - \bar{m}_{\text{NF},r,j}$, which explicitly measures how strongly a region-feature pair distinguishes hallucination failures from other general failures. For the forget set $\mathcal{F}$, we rank region-feature pairs using an aggregated score $s_{r,j}^{\mathcal{F}} = s_{r,j}^{\text{hall}} + s_{r,j}^{\text{sep}} - s_{r,j}^{\text{nom}}$. This score prioritizes features that are strongly associated with

TABLE I: Performance comparison in terms of hallucination and task success. ‘Hall.’ denotes hallucination.

Method	Hallucination ↓	Harmful hallucination ↓	Overall success ↑	Clean success ↑	Grasp Hall. ↓	Transport Hall. ↓	Place Hall. ↓
LIBERO-Plus with OpenVLA							
Original Policy [2]	70.0	60.0	26.0	16.0	70.0	38.0	16.0
Gradient Ascent [27]	84.0	84.0	0.0	0.0	84.0	62.0	0.0
SOUL (Ours)	44.0	38.0	58.0	52.0	44.0	20.0	6.0
RoboCasa with $\pi_{0.5}$							
Original Policy [3]	40.0	37.1	41.9	39.0	39.0	7.6	1.9
Gradient Ascent [27]	60.0	59.0	4.8	3.8	60.0	0.0	0.0
SOUL (Ours)	22.9	21.0	54.3	52.4	22.9	3.8	0.0

hallucination failures, distinguishable from non-hallucination failures, and weakly associated with successful execution. We select the top-ranked region-feature pairs as the forget set $\mathcal{F}$. For the retain set $\mathcal{R}$, we select region-feature pairs with high $s_{r,j}^{\text{nom}}$. The resulting forget and retain sets specify which internal representations should be suppressed and preserved during policy unlearning.

Feature-guided policy unlearning. Given the forget set $\mathcal{F}$ and retain set $\mathcal{R}$, we update the VLA to suppress hallucination-associated features while preserving nominal-associated features. During unlearning, the SAE encoder is frozen and used only to compute sparse feature activations, while the VLA policy θ is updated using the proposed unlearning objective. For hallucination failure samples, we minimize the activation of the selected forget features as

$$\mathcal{L}_{\text{forget}} = \mathbb{E}_{x \in \mathcal{D}_{\text{HF}}} \left[\frac{1}{|\mathcal{F}|} \sum_{(r,j) \in \mathcal{F}} (z_{r,j}(x; \theta))^2 \right]. \quad (4)$$

For clean success samples, we encourage the selected retain features to remain active as

$$\mathcal{L}_{\text{retain}} = -\mathbb{E}_{x \in \mathcal{D}_{\text{CS}}} \left[\frac{1}{|\mathcal{R}|} \sum_{(r,j) \in \mathcal{R}} z_{r,j}(x; \theta) \right]. \quad (5)$$

The final objective jointly balances forgetting and retention as

$$\mathcal{L}_{\text{unlearn}} = \mathcal{L}_{\text{forget}} + \lambda_r \mathcal{L}_{\text{retain}}, \quad (6)$$

where λ_r controls the strength of retention.

V. EXPERIMENTS

A. Setup

Evaluation protocol. We conduct experiments with the autoregressive OpenVLA [2] and flow-matching-based $\pi_{0.5}$ [3] in LIBERO-Plus [9], RoboCasa [14], and a real-world setup with a 7-DoF Franka Research 3 robotic arm equipped with a two-finger parallel gripper. LIBERO-Plus provides realistic visual perturbations at multiple severity levels, while RoboCasa provides diverse kitchen environments and everyday manipulation tasks. To provide a controlled setting for evaluating state hallucination while minimizing confounding effects from task and motion planning, we focus on pick-and-place tasks, a fundamental manipulation primitive underlying many complex

tasks, with a structured grasp-transport-place sequence.¹ To our knowledge, no prior work has established an unlearning baseline for robot policies. For a direct comparison from an unlearning perspective, we therefore adapt gradient ascent, a commonly used optimization-based unlearning baseline [27], which maximizes the action prediction loss on hallucination failure samples.

SAE-based feature analysis and unlearning. We use Top- k SAEs [33], [34] to decompose visual-token activations into sparse latent features, whose fixed number of active features provides a consistent basis for comparing activation patterns across executions. The SAE objective combines the standard reconstruction loss with an auxiliary reconstruction loss using the top- k_{aux} inactive features to provide additional learning signals for rarely activated features. We set the SAE latent dimension to $d = 4096$, the number of active features to $k = 100$ and $k_{\text{aux}} = 512$. Using the resulting sparse activations, we perform region-conditioned feature analysis and select the top five hallucination-associated and nominal-associated region-feature pairs as forgetting and retention targets, respectively. For policy unlearning, we initialize from the publicly released pretrained policy and optimize LoRA adapters [35] with our unlearning objective while keeping the original model weights frozen. We uniformly sample up to 20 time steps per rollout to reduce bias toward longer trajectories. For feature analysis, we use layers 0, 8, 16, 24, and 31 of the OpenVLA language backbone and layers 0, 5, 11, and 17 of the PaliGemma backbone in $\pi_{0.5}$.

Metrics. To evaluate task performance and state hallucination, we report rollout and event levels metrics. *Hallucination* and *harmful hallucination* are rollout-level metrics, measuring the fraction of rollouts containing at least one state hallucination event and the fraction of rollouts that both contain state hallucination and result in task failure, respectively. *Overall success* and *clean success* are task-level metrics, measuring task completion and task completion without state hallucination, respectively. We further report hallucination

¹For LIBERO-Plus, we use the base tasks *put the bowl on the stove*, *pick up the black bowl on the cookie box* and *place it on the plate*, and *pick up the tomato sauce and place it in the basket* from the goal, spatial, and object suites, respectively, together with their corresponding lighting and background perturbed evaluation variants. For RoboCasa, we use the task families *PickPlaceCounterToStandMixer*, *PickPlaceDrawerToCounter*, *PickPlaceCounterToSink*, *PickPlaceCabinetToCounter*, *PickPlaceCounterToCabinet*, and *PickPlaceFridgeDrawerToShelf*, together with their corresponding evaluation configurations.

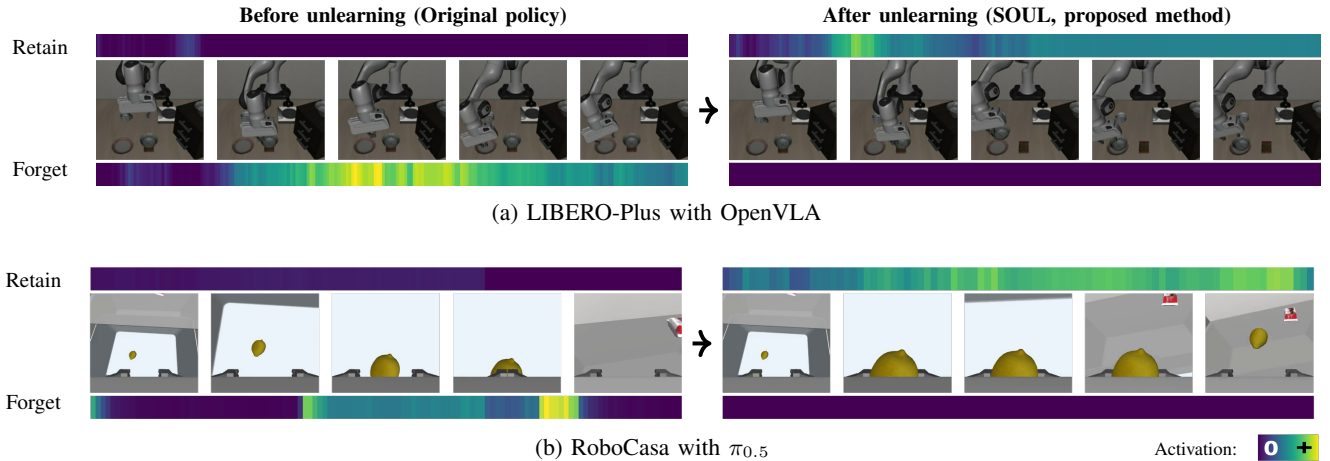


Fig. 5: **Visualizations of rollout behavior and feature activations on benchmarks.** We compare the original policy and SOUL, showing the temporal activations of selected forget and retain features during execution.

rates for *grasp*, *transport*, and *place*, defined as the fraction of rollouts containing at least one hallucination of each type.

B. Main Results

Quantitative results. To quantitatively evaluate whether our proposed sparse feature policy unlearning (SOUL) improves policy reliability by reducing state hallucination without compromising task execution, we compare three policies: the original pretrained policy, gradient-ascent unlearning, and SOUL. As shown in TABLE I, the original policies exhibit average hallucination and harmful hallucination rates of 55.0% and 48.6%, respectively. Gradient-ascent unlearning does not consistently mitigate hallucination and substantially degrades task success, suggesting that directly forgetting hallucination samples can interfere with useful manipulation capabilities. In contrast, SOUL reduces the average hallucination rate from 55.0% to 33.5%, while increasing clean success from 27.5% to 52.2%. Hallucination rates of the grasp, transport, and place stages are also consistently reduced, showing that the improvement extends across different stages of manipulation rather than being limited to a particular stage. These results show that SOUL improves policy reliability by selectively mitigating recurring state hallucination behaviors while preserving useful manipulation capabilities.

Qualitative results. To qualitatively examine how SOUL modifies sparse feature activations during execution, Fig. 5 compares rollout behaviors and the temporal activations of selected forget and retain features before and after applying our unlearning method, SOUL, across both VLAs and benchmarks. Before unlearning, the original policies exhibit state hallucination, with strong forget feature activation emerging around the hallucination event while retain feature activation remains weak. After unlearning, the policies successfully complete the tasks without state hallucination, accompanied by suppressed forget feature activation and increased retain feature activation during the corresponding manipulation stages. This pattern is consistently observed for OpenVLA on LIBERO-Plus and $\pi_{0.5}$ on RoboCasa. These results show that

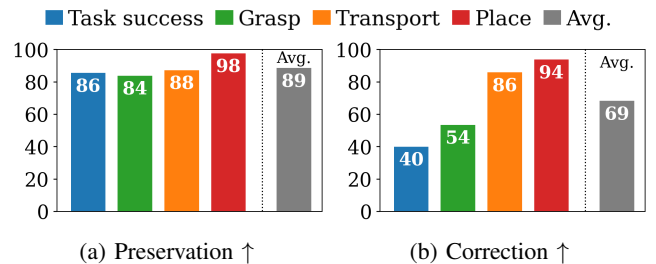


Fig. 6: **Behavior preservation and correction after applying SOUL.** (a) Preservation measures the fraction of originally successful task executions that remain successful and non-hallucinated events that remain non-hallucinated after applying SOUL. (b) Correction measures the fraction of originally failed task executions that become successful and hallucinated events that become non-hallucinated after applying SOUL. Higher values indicate better preservation or correction.

the objective of SOUL modifies the selected sparse features in the intended direction, suppressing hallucination-associated activations while promoting those associated with clean successful execution, together with a shift from hallucinated failures to successful behavior.

C. More Results

Behavior preservation and correction. To evaluate the unlearning trade-off between correcting undesirable hallucination behaviors and preserving desirable behaviors, Fig. 6 reports preservation and correction at both task and event levels. SOUL preserves 89% of originally successful task executions and non-hallucinated events on average. Meanwhile, it converts 40% of failed task executions into successful ones and corrects 78% of hallucinated events across the grasp, transport, and place stages. These results demonstrate a favorable unlearning trade-off, substantially correcting hallucination-related undesirable behaviors while largely preserving behaviors that were already successful.

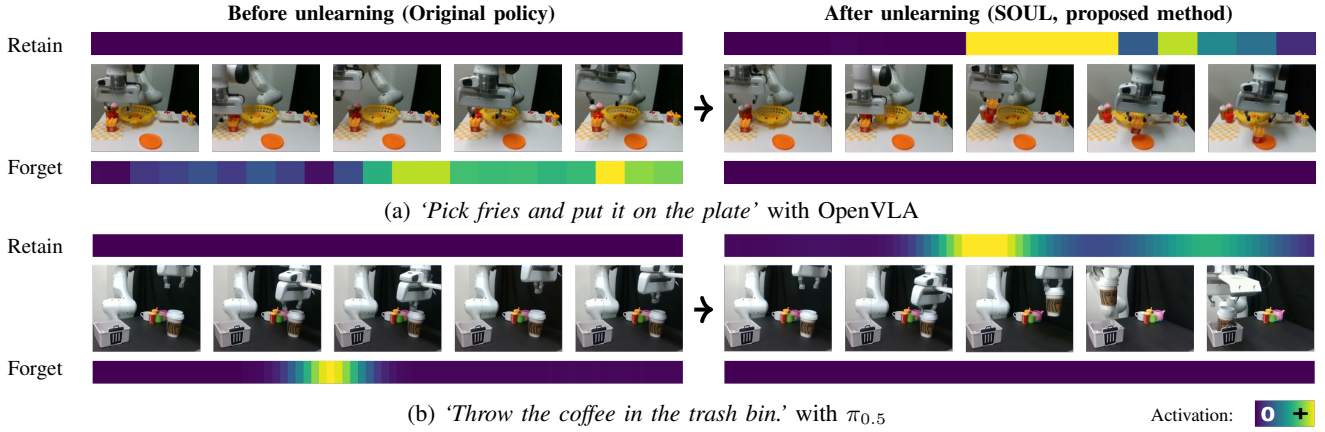


Fig. 7: **Real-world rollouts and selected sparse feature activations before and after applying the proposed unlearning, SOUL.** We compare the original policy and SOUL across representative real-world rollouts spanning both models and tasks.

TABLE II: **Comparison between global, tokenized, and region-guided feature policy unlearning.** ‘Global’ pools activations across all visual tokens, ‘Tokenized’ selects features at the visual-token level, and ‘Region-Guided’ selects features using semantic-region-conditioned activations.

Method	Hall. ↓	Harmful hall. ↓	Overall success ↑	Clean success ↑
Global	45.8	41.4	42.7	38.3
Tokenized	41.5	35.5	47.3	41.3
Region-Guided	33.5	29.5	56.2	52.2

Effect of region guidance. To evaluate the effect of spatial grounding on sparse feature policy unlearning, we compare global, tokenized, and region-guided feature selection in TABLE II, with results averaged across LIBERO-Plus and RoboCasa. Global feature selection provides only limited improvement, likely because pooling across visual tokens removes spatial information needed to isolate localized hallucination-associated patterns. Nevertheless, it still improves over the original policy, showing that sparse feature policy unlearning can be effective even without explicit spatial guidance. Tokenized selection further improves performance by preserving token-level localization during feature selection. Region-guided selection performs best, achieving the lowest hallucination rates and the highest overall and clean success rates by grounding feature selection to task-relevant regions. These results show that spatial guidance progressively improves the selectivity of sparse feature policy unlearning, with region-guided selection providing the most effective suppression of hallucination-associated features while preserving successful execution.

Effect of score aggregation and retain objective.

TABLE III ablates score aggregation and the retain objective as defined in Section IV, with results averaged across LIBERO-Plus and RoboCasa. Removing the retain objective in the second row still reduces hallucination, but lowers clean success relative to the full method in the first row, indicating that the retain objective is important for preserving successful behavior. Removing score aggregation in the

TABLE III: **Ablation study of score aggregation and the retain objective.** The aggregated score $s_{r,j}^{\text{agg}}$ combines $s_{r,j}^{\text{sep}}$ and $-s_{r,j}^{\text{clean}}$ for forget target selection.

$s_{r,j}^{\text{sep}}$	$s_{r,j}^{\text{agg}}$	$\mathcal{L}_{\text{retain}}$	Hall. ↓	Harmful hall. ↓	Overall success ↑	Clean success ↑
✓	✓	✓	33.5	29.5	56.2	52.2
✓	✓		36.8	29.9	55.2	48.3
✓		✓	39.9	36.9	50.7	47.7

third row degrades both hallucination reduction and task success, suggesting that the raw forget score alone yields less selective forgetting targets. The full method, combining score aggregation with the retain objective, achieves the best balance between hallucination reduction and task success. Overall, score aggregation improves the selectivity of forget target selection, while the retain objective helps preserve successful manipulation behavior.

Real-world evaluation. We further evaluate SOUL on a Franka Research 3 robot using OpenVLA and $\pi_{0.5}$, conducting 50 rollouts across the both models under diverse visual conditions. Starting from the publicly released pretrained models, we first fine-tune each model for its target task using LoRA and then apply SOUL to the task-specific policies. As shown in Fig. 7, the original policies exhibit state hallucination, accompanied by strong activation of selected forget features and weak activation of retain features. After applying SOUL, the policies complete the tasks without state hallucination, with suppressed forget-feature activations and increased retain-feature activations. Quantitatively, SOUL increases clean success by an average of 20% while reducing harmful hallucination by 22% across the all settings. These results show that SOUL transfers beyond simulation, effectively mitigating state hallucination in physical robot execution under diverse real-world visual conditions.

VI. CONCLUSION

We have presented a study of selective policy unlearning in VLAs, targeting state hallucination as a recurring undesirable execution behavior. Our analysis showed that state

hallucination is accompanied by weakened attention to task-relevant visual regions and distinct region-conditioned sparse feature patterns associated with hallucinated executions. Based on these findings, we proposed *SOUL*, which selectively unlearns policy knowledge associated with state hallucination by suppressing hallucination-associated region-feature pairs while preserving nominal-associated ones. Experiments across VLA architectures in simulated and real-world environments show that *SOUL* reduces state hallucination and improves task success while largely preserving existing manipulation capabilities. These results show that interpretable sparse feature analysis can move beyond diagnosing VLA failures to provide actionable targets for policy unlearning, enabling selective modification of recurring undesirable behaviors in embodied robot policies.

REFERENCES

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair *et al.*, “Openvla: An open-source vision-language-action model,” in *Conference on Robot Learning*. PMLR, 2025, pp. 2679–2713.
- [3] K. Black *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 17–40.
- [4] Open X-Embodiment Collaboration *et al.*, “Open x-embodiment: Robotic learning datasets and RT-X models,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6892–6903.
- [5] NVIDIA *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” in *ArXiv Preprint*, March 2025.
- [6] Gemini Robotics Team *et al.*, “Gemini robotics: Bringing ai into the physical world,” *arXiv preprint arXiv:2503.20020*, 2025.
- [7] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, and F. Shkurti, “Safe: Multitask failure detection for vision-language-action models,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 40 041–40 076, 2026.
- [8] B. Zhang, Y. Zhang, J. Ji, Y. Lei, J. Dai, Y. Chen, and Y. Yang, “Safevla: Towards safety alignment of vision-language-action model via constrained learning,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 153 335–153 373, 2026.
- [9] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei *et al.*, “Libero-plus: In-depth robustness analysis of vision-language-action models,” *arXiv preprint arXiv:2510.13626*, 2025.
- [10] X. Zhou, Y. Xu, G. Tie *et al.*, “Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization,” *arXiv preprint arXiv:2510.03827*, 2025.
- [11] Y. Zhang, Y. Qi, and X. Zheng, “Experiences from benchmarking vision-language-action models for robotic manipulation,” *arXiv preprint arXiv:2511.11298*, 2025.
- [12] Z. Lin, J. Duan, H. Fang *et al.*, “Failsafe: Reasoning and recovery from failures in vision-language-action models,” *arXiv preprint arXiv:2510.01642*, 2025.
- [13] H. Soh and E. Lim, “Action hallucination in generative visual-language-action models,” *arXiv preprint arXiv:2602.06339*, 2026.
- [14] X. Zhou, Y. Xu, G. Tie *et al.*, “RoboCasa: Large-scale simulation of household tasks for generalist robots,” in *Robotics: Science and Systems XX*, 2024.
- [15] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu *et al.*, “Palm-e: an embodied multimodal language model,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 8469–8488.
- [16] D. Ghosh *et al.*, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [17] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [18] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti *et al.*, “Smolvla: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [19] Q. Zhao, Y. Lu, M. J. Kim, Z. Fu, Z. Zhang, Y. Wu, Z. Li, Q. Ma, S. Han, C. Finn *et al.*, “Cot-vla: Visual chain-of-thought reasoning for vision-language-action models,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 1702–1713.
- [20] F. Lin, R. Nai, Y. Hu, J. You *et al.*, “OnetwoVLA: A unified vision-language-action model with adaptive reasoning,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [21] C. Liu, W. Tan, L. Zhu, F. Li, J. Li, G. Yang, and H. T. Shen, “Self-correcting vla: Online action refinement via sparse world imagination,” *arXiv preprint arXiv:2602.21633*, 2026.
- [22] J. Kwok, C. Agia, R. Sinha, M. Foutter, S. Li, I. Stoica, A. Mirhoseini, and M. Pavone, “Robomongo: Scaling test-time sampling and verification for vision-language-action models,” in *Conference on Robot Learning*. PMLR, 2025, pp. 3200–3217.
- [23] S. Jang, D. Kim, C. Kim, Y. Kim, and J. Shin, “Verifier-free test-time sampling for vision-language-action models,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [24] B. H  on, K. C. Stocking, I. Chuang, and C. Tomlin, “Mechanistic interpretability for steering vision-language-action models,” in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 2743–2762.
- [25] M. A. Khan, N. Boskov, F. M. Anwar, and M. A. Khan, “Controlling vision–language–action policies through sparse latent directions,” in *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025.
- [26] Y. Cao and J. Yang, “Towards making systems forget with machine unlearning,” in *2015 IEEE symposium on security and privacy*. IEEE, 2015, pp. 463–480.
- [27] A. Golatkar, A. Achille, and S. Soatto, “Eternal sunshine of the spotless net: Selective forgetting in deep networks,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2020, pp. 9301–9309.
- [28] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, “Machine unlearning,” in *2021 IEEE symposium on security and privacy (SP)*. IEEE, 2021, pp. 141–159.
- [29] T. T. Nguyen, T. T. Huynh, Z. Ren, P. L. Nguyen, A. W.-C. Liew, H. Yin *et al.*, “A survey of machine unlearning,” *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 5, pp. 1–46, 2025.
- [30] Y. Yao and X. Xu, “Large language model unlearning,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 105 425–105 475, 2024.
- [31] P. Maini, Z. Feng, A. Schwarzschild, Z. C. Lipton, and J. Z. Kolter, “Tofu: A task of fictitious unlearning for llms,” *arXiv preprint arXiv:2401.06121*, 2024.
- [32] R. Huben, H. Cunningham, L. R. Smith, A. Ewart, and L. Sharkey, “Sparse autoencoders find highly interpretable features in language models,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [33] L. Gao, T. Dupre la Tour, H. Tillman, G. Goh, R. T. Roll, A. Radford, I. Sutskever, J. Leike, and J. Wu, “Scaling and evaluating sparse autoencoders,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 26 721–26 754.
- [34] A. Makhzani and B. Frey, “K-sparse autoencoders,” *arXiv preprint arXiv:1312.5663*, 2013.
- [35] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.